



OPEN Towards intelligent edge computing through reinforcement learning based offloading in public edge as a service

Ateeqa Jalal¹, Umar Farooq^{1,2,3}, Ihsan Rabbi^{1,3}, Afzal Badshah⁵✉, Aurangzeb Khan^{1,6}, Muhammad Mansoor Alam^{4,7} & Mazliham Mohd Su'ud⁴✉

Internet of Things (IoT) deployments face increasing challenges in meeting strict latency and cost requirements while ensuring efficient resource utilization in distributed environments. Traditional offloading often overlooks the role of intermediate regional layers and mobility, resulting in inefficiencies in real-world deployments. To address this gap, we propose Public Edge as a Service (PEaaS) as an intermediate tier and develop *RegionalEdgeSimPy*, a Python simulator to model and evaluate this framework. It uses a Proximal Policy Optimization (PPO) scheduler that models mobility and considers multiple input parameters (e.g., network latency, cost, congestion, and energy). Tasks are first evaluated at the serving (Wireless Access Point (WAP)) for feasibility under utilization thresholds. This decision uses action masking to restrict invalid options, and a reward function that integrates latency, cost, congestion, and energy to guide optimal offloading. Simulations conducted with 10 to 3000 devices in a 10×10 Kilometers smart city area. Results show that PPO prioritizes Edge processing until over-utilization, after which workloads are offloaded to the nearest PEaaS, with Cloud used sparingly. On average, Edge achieves 75.8% utilization, PEaaS stabilizes near 52.9%, and Cloud remains under 1.2% when active. These findings demonstrate that the PPO scheduling significantly reduces delay, cost, and task failures, providing improved scalability for mobility in IoT big data processing.

Keywords Edge computing, Cloud computing, Public edge as a service, Reinforcement learning

The rapid growth of IoT devices, smart environments, and digital services has led to massive data generation, with global data volumes projected to exceed 291 zettabytes by 2027¹. This increase in big data imposes critical challenges in processing, storage, and communication infrastructures. These require solutions to ensure scalability, cost efficiency, and ultra-low latency for time-sensitive applications. Traditional cloud computing has scalable computational resources; however, suffers from long-distance communication, high latency, and congestion, making it unsuitable for real-time workloads^{2,3}. Edge computing addresses latency by processing data closer to the source, yet its limited scalability and high ownership costs resist large-scale adoption. Hybrid paradigms (e.g., fog computing, cloudlets, and Mobile Edge Computing (MEC)) attempt to balance performance and cost; however, they face resource constraints^{4,5}.

To address these challenges, our earlier work introduced PEaaS, a collaborative framework that utilizes free computational resources contributed by universities, enterprises, and individuals under a service-sharing model^{6,7}. By decentralizing computation and enabling multi-tenant resource utilization, PEaaS significantly reduced latency, communication costs, and task failures compared to traditional cloud approaches. We evaluated PEaaS using EdgeCloudSim and confirmed that it delivers near-edge performance with cost-effective scalability,

¹Institute of Computing Science and Information Technology (ICSIT), University of Science and Technology, Bannu (UST Bannu), Bannu, Pakistan. ²Heriot-Watt International Faculty, K. Zhubanov University, Aktobe, Kazakhstan. ³Centre of Excellence for Artificial Intelligence, Multimedia University, Cyberjaya, Malaysia. ⁴Faculty of Computing, Multimedia University, Cyberjaya, Malaysia. ⁵Department of Software Engineering, University of Sargodha, Sargodha, Pakistan. ⁶University of Lakki Marwat, Lakki Marwat, Khyber Pakhtunkhwa, Pakistan. ⁷Riphah International University, Islamabad, Pakistan. ✉email: afzalbadshahkhattak@gmail.com; mazliham@mmu.edu.my

offering a sustainable alternative to centralized cloud infrastructures. These findings highlight the promise of PEaaS in bridging the gap between cloud and edge computing for next-generation big data applications.

However, the original PEaaS framework depend on static threshold rules to guide offloading across Edge, PEaaS, and Cloud. While this rule driven design achieved performance gains under controlled scenarios, it lacked adaptability to real-world conditions characterized by dynamic workloads, fluctuating bandwidth, and heterogeneous task requirements. Such deterministic policies cannot respond effectively to congestion variations, shifting resource availability, or task priorities, leading to inefficiencies in resource utilization and degraded quality of service^{8,9}. This limitation shows the urgent need for adaptive and intelligent offloading frameworks that can optimize performance under complex and dynamic environments.

Reinforcement Learning (RL) provides adaptive decisions by learning from interactions with the environment¹⁰. Unlike fixed threshold strategies, RL schedulers incorporate real-time feedback on latency, cost, congestion, and utilization, enabling intelligent policies that evolve with changing conditions. PPO, in particular, has gained prominence for its stable learning and efficiency in dynamic resource allocation problems^{11,12}. Integrating RL into PEaaS enables adaptive scheduling, replacing static offloading and ensuring balanced, priority-aware allocation across multi-tier infrastructures¹³.

In this paper, we propose a RL based decision-making framework for the offloading process. The scheduler that dynamically selects the optimal tier (e.g., Edge, PEaaS, or Cloud) based on latency, congestion, cost, and resource utilization. This approach overcomes the rigidity of threshold based models and adapts to mobility variations and real-time network dynamics. Furthermore, we code *RegionalEdgeSimPy*, a Python simulator that models multi-tier infrastructures, integrates mobility dynamics, and supports RL schedulers. Unlike earlier Java platforms such as *EdgeCloudSim*, our simulator is extensible and supports easy Artificial Intelligence (AI) integration.

The main contributions of this paper are summarized as follows:

- 1) We introduce the Public Edge as a Service (PEaaS) tier, a publicly accessible intermediate computing layer between Edge and Cloud. This new tier reshapes the edge–cloud continuum and enables scalable, low-cost, and mobility-aware task execution beyond conventional two-tier architectures.
- 2) We propose a mobility-aware, multi-tier scheduling framework that incorporates feasibility-first filtering, hierarchy-aware tier selection, and deterministic server mapping based on utilization thresholds and proximity. This produces a practical and robust decision pipeline suitable for dynamic smart-city environments.
- 3) We develop a PPO-based reinforcement learning scheduler that operates on a structured global state capturing CPU load, memory usage, congestion level, cost, propagation delay, energy consumption, and mobility-related indicators across all computing tiers. This enables real-time adaptation to highly variable workloads and network conditions.
- 4) We build *RegionalEdgeSimPy*, an extensible Python simulation platform that models mobility, multi-tier resource interactions, dynamic utilization dynamics, and AI-enabled scheduling. It supports large-scale smart-city scenarios and provides a reproducible environment for evaluating the proposed PEaaS framework.

The rest of this paper is organized as follows. In Section “Literature”, we review the related work. Section “Methodology” presents the system and decision model, covering the RL scheduling framework. The evaluation, experimentation, and results are covered in Section “Evaluation”. Critical analysis is discussed in Section “Discussion”. Finally, Section “Conclusion” concludes the paper and outlines potential future research directions.

Literature

The rapid expansion of smart city infrastructures, where heterogeneous devices continuously generate massive data, has created a high demand for more efficient computation offloading. Though cloud computing offers scalable resources, it suffers from backhaul latency and congestion, making it unsuitable for latency-critical urban services (e.g., real-time surveillance and emergency response)^{14,15}. Consequently, recent studies explore distributed and adaptive paradigms that place computation closer to the data sources^{16,17}.

Rule-based offloading

Early approaches relied on static thresholds and heuristic policies that steered tasks among devices, nearby servers, and the cloud; based on pre-set rules^{18,19}. These strategies are simple; however, become harder under dynamic traffic and time-varying conditions, which lead to suboptimal performance²⁰. While local execution can avoid communication delay, the computing and memory constraints of IoT devices rapidly become limiting for data-intensive analytics, motivating external offloading once on-device resources are overutilized^{21,22}. Priority has been introduced to differentiate latency-sensitive applications (for example, urgent alerts versus periodic monitoring), however, most policies remain unspecific and slow to adapt when conditions shift^{23,24}.

Hierarchical offloading

Multi-tier designs combine edge resources at access points or gateways with regional servers and cloud data centers to balance latency and scalability^{4,15}. Edge servers reduce delay, while regional servers provide intermediate capacity to relieve hotspots before migrating to the cloud^{20,25}. However, the mobility of devices, fluctuating demand, and unbalance server utilization complicate the provision of consistently low-latency

service. Popular access points can become congested, and available servers can change rapidly^{8,26}. Although the cloud remains essential for heavy workload, its distance makes it ill-suited for hard real-time constraints²⁷. Hybrid strategies that split workloads across tiers have been explored; however, coordination overheads, load prediction, and explicit priority or quality-of-service differentiation are still not addressed^{17,28}.

Optimization based offloading

Metaheuristics such as Particle Swarm Optimization and Genetic Algorithms have been applied to offloading and task allocation in IoT and smart city environments, offering effective global search over large solution spaces^{1,16,29}. However, these methods typically require iterative, often offline computation, limiting responsiveness in fast-changing urban environments^{30,31}. Hybridization with adaptive learning has been proposed to improve agility, however, real-time feasibility at the city scale remains challenging^{30,31}.

Learning based offloading

RL enables data-driven and context-aware offloading policies that adapt to varying workload, link quality, and resource availability^{15,32,33}. Deep RL, including DQN and actor-critic methods, has demonstrated effectiveness in complex, dynamic environments^{34,35}. Mobility-aware extensions use device trajectories and connection durations to improve decision robustness during movement and handovers^{36–38}. Despite progress, many RL systems optimize a single objective (e.g., latency or energy). This approach offers limited treatment of congestion, server utilization, and explicit task priority within the control loop, which are critical for smart city service differentiation^{39–41}.

Challenges and research gaps

The existing research simplifies the decision space to only edge or cloud, however, overlooked the intermediate regional servers that can absorb bursty demand and reduce congestion^{25,42}. While multi-objective formulations exist, unifying delay, cost, and energy within a tractable, real-time policy is still uncommon^{20,43}. Methods that jointly collect live network feedback (e.g., link quality, congestion) and compute state (e.g., CPU, memory, storage utilization) remain limited^{32,44,45}. Most importantly, priority-aware scheduling, where task criticality directly shapes server selection and resource reservation, remains underexplored across both heuristic and learning-based approaches⁴⁶. To address these gaps, our paper introduces PEaaS, a paradigm that offers edge resources as a shared, scalable service layer for smart city devices. By combining PEaaS with PPo scheduling, our work addresses the shortcomings of existing approaches and supports quality of service differentiation in dynamic smart city environments.

Methodology

This section covers a comprehensive explanation of our proposed methodology, as shown in Fig. 1. For clarity and consistency, Table 1 summarizes the mathematical symbols and parameters used throughout the system model and RL formulations.

System architecture

We consider a hierarchical multi-tier computing topology over a smart urban region spanning 10×10 Kilometers. The architecture comprises three interconnected layers (Edge, PEaaS, and Cloud) denoted by the sets $\mathbb{E}$, $\mathbb{P}$, and $\mathbb{C}$, respectively. These layers are collectively referred to as the total server set $\mathbb{T}$.

$$\mathbb{T} = \mathbb{E} \cup \mathbb{P} \cup \mathbb{C} \quad (1)$$

Each tier contains multiple server instances:

$$\mathbb{E} = \{E_1, E_2, \dots, E_N\} \quad (2)$$

$$\mathbb{P} = \{P_1, P_2, \dots, P_N\} \quad (3)$$

$$\mathbb{C} = \{C_1, C_2, \dots, C_N\} \quad (4)$$

where E_N , P_N , and C_N represent the number of Edge, PEaaS, and Cloud nodes deployed across the infrastructure.

Each server $s \in \mathbb{T}$ is modeled using the resource vector:

$$s = \langle C_s, M_s, S_s, B_s, L_s, \kappa_s \rangle \quad (5)$$

where C_s (MIPS), M_s (GB), and S_s (GB) denote the CPU, memory, and storage capacities; B_s is the available bandwidth (Mbps); L_s is the average one-way communication latency (ms); and κ_s is the unit processing cost (\$/MIPS).

To ensure consistent tier-based ordering, the global list of servers is computed as:

$$S = \text{sort}(\mathbb{E}) \circ \text{sort}(\mathbb{P}) \circ \text{sort}(\mathbb{C}) \quad (6)$$

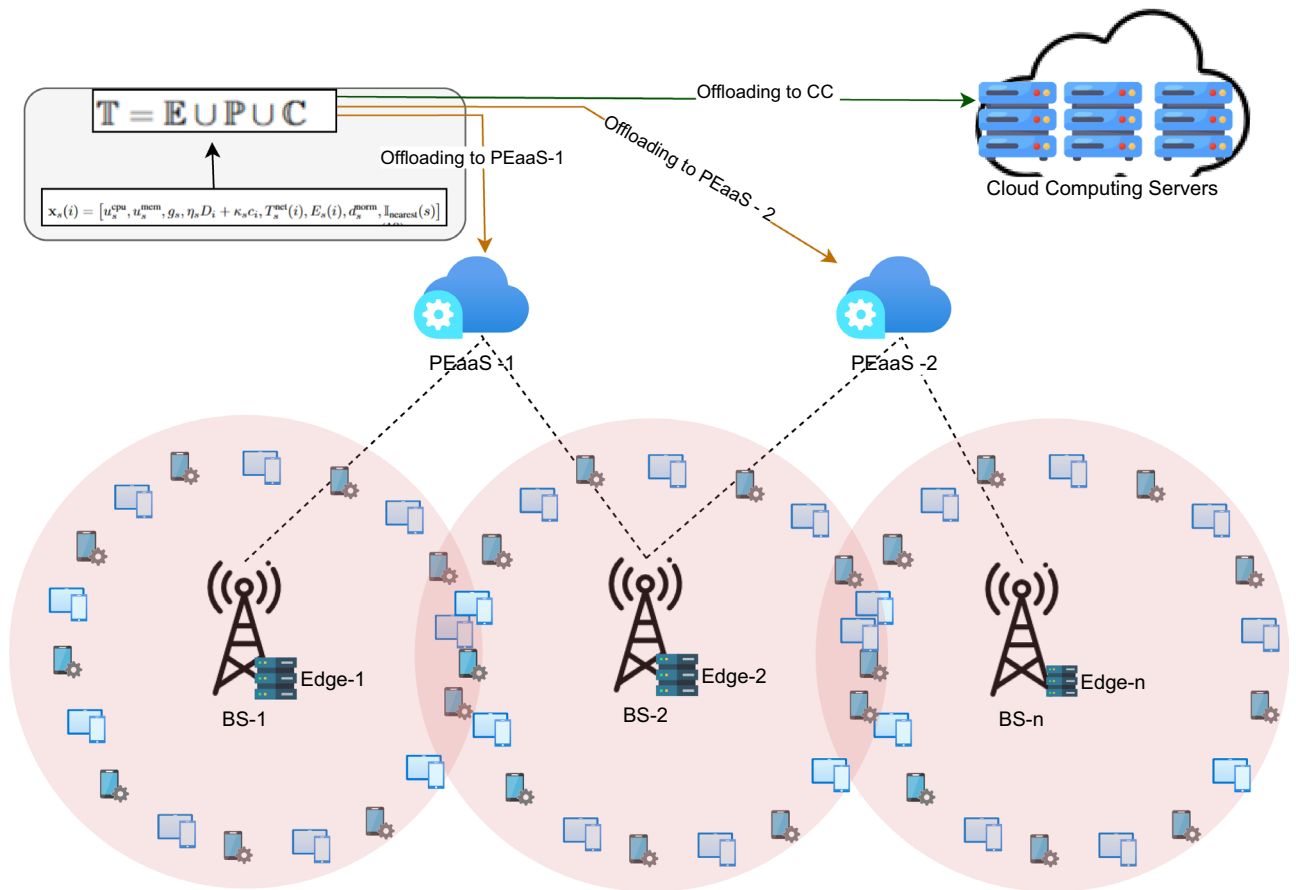


Fig. 1. Proposed PEaaS framework for hierarchical task offloading in edge, PEaaS, and cloud environments.

where $\circ$ represents the concatenation operation on the list. This order will be constant over all simulation steps of the simulation process in `_order_servers()`, providing deterministic access to capabilities in the PPO.

Edge servers are tightly coupled with WAP and offer low-latency processing ($L_E \approx 5$ ms); however, have non-scalable resources ($C_E = 2.8 \times 10^5$ MIPS). PEaaS geographically distributed across the city, have a higher capacity ($C_P = 6.4 \times 10^6$ MIPS) with moderate latency ($L_P \approx 50$ ms). Cloud datacenters provide scalable resources ($C_C = 10^8$ MIPS); however, they show higher latency ($L_C \approx 300$ ms) due to long communication routing.

The scheduling hierarchy follows a bottom-up strategy. Tasks are first offered to the nearest WAP if feasible; otherwise, offloaded to the closest admissible PEaaS. When both Edge and PEaaS are infeasible, Cloud is used as a fallback, prioritized by minimum propagation delay. This structured preference is enforced through reward shaping in the PPO model to avoid suboptimal Cloud placements when lower tiers are available.

Mobility and communication model

Devices in the simulation follow a randomized mobility pattern across the $10 \text{ km} \times 10 \text{ km}$ city grid, dynamically associating with the nearest available WAP. These WAP are located with Edge servers, and the association switches as the device moves.

The total network delay faced when transmitting a task from a device to a server s is defined as:

$$T_s^{\text{net}} = T_s^{\text{tx}} + T_s^{\text{prop}} + T_s^{\text{ho}} \quad (7)$$

The transmission delay is modeled as:

$$T_s^{\text{tx}} = \frac{D}{B_s} \quad (8)$$

Notation	Description	Notation	Description
$\mathbb{E}, \mathbb{P}, \mathbb{C}$	Sets of Edge, PEaaS, and Cloud servers	$s \in \mathbb{T}$	A server in the set $\mathbb{T} = \mathbb{E} \cup \mathbb{P} \cup \mathbb{C}$
C_s, M_s, St_s	CPU, memory, and storage capacity of server s	$B_s, L_s, \kappa_s, \eta_s$	Bandwidth, latency, processing cost, and transmission cost of s
P_s, P_{WP}	Positions of server s and device's wireless point	d_s^{norm}	Distance from WAP to s normalized by city diagonal
$\mathcal{D}$	Set of mobile devices	$p_i(t), v_i(t)$	Position and velocity of device i at time t
$P_{\text{RSS}}(d)$	Received signal power at distance d	θ_{ho}	Threshold for handover (in dB)
$T^{\text{prop}}, T^{\text{tx}}, T^{\text{ho}}, T^{\text{net}}$	Propagation, transmission, handover, and total delay	v_w	Speed of wireless propagation (3×10^8 m/s)
τ_i	Task i	D_i, c_i, m_i, st_i	Data size, CPU, memory, and storage demand of task i
s_E	Device's own edge server	s^*	Selected offloading server
$u_s^{\text{cpu}}, u_s^{\text{mem}}$	CPU and memory utilization of server s	g_s	Congestion level of server s
$E_s(i)$	Estimated energy for processing task i at s	u^*	Target utilization level for shaping reward
$\mathbf{x}_s(i)$	Feature vector of server s for task i	$\mathbf{s}_i$	Global state vector for task i
ℓ_i	Action logits vector (PEaaS or Cloud)	V_i	Critic-estimated value for task i
W_1, W_2, W_π, W_v	Neural network weights	$\mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_\pi, \mathbf{b}_v$	Corresponding bias vectors
$\hat{A}_t$	Advantage estimate at time t	$r_t(\theta)$	Policy ratio between new and old policies
ϵ	PPo clipping parameter	λ_V, λ_H	Weights for value loss and entropy regularization
R_i, R_t	Task reward and return at time t	$\mathcal{L}(\theta)$	PPo loss function
$\bar{T}_{\text{net}}, \bar{C}, \bar{E}$	Average delay, cost, and energy	Failure Rate	Fraction of tasks not scheduled successfully
$\bar{u}^{\text{cpu}}, \sigma^{\text{cpu}}$	Mean and std. dev. of CPU utilization	RUB	Resource Utilization Balance
$R_i^{\text{scaled}}, R_i^{\text{clipped}}$	Scaled and clipped reward for PPo stability		

Table 1. System parameters and notation.

where D is the task data size and B_s is the bandwidth available at the server s (in Mbps).

The propagation delay depends on the Euclidean distance between the WAP and server s , divided by the wireless signal propagation speed v (assumed as 3×10^8 m/s):

$$T_s^{\text{prop}} = \frac{d_s}{v} \quad (9)$$

Handover delay is modeled as a fixed penalty when a new WAP provides a stronger signal:

$$T_s^{\text{ho}} = \mathbb{I}_{\text{ho}} \cdot \delta_{\text{ho}} \quad (10)$$

where $\mathbb{I}_{\text{ho}}$ is an indicator function (1 if handover occurs, 0 otherwise), and δ_{ho} is a constant penalty (e.g., 5 ms).

Additionally, signal strength is estimated using a simplified path-loss model:

$$P_{\text{RSS}}(d) = P_0 - 20 \log_{10}(d) \quad (11)$$

where d is the distance to the WAP, and P_0 is the reference signal power measured at one meter. A handover is triggered when a new WAP provides at least $\theta_{\text{ho}} = 3$ dB greater signal strength than the currently connected WAP.

These mobility-aware communication metrics directly influence tier selection, task placement, and reward formulation within the PPo scheduling policy.

Task and scheduling workflow

Each incoming task is represented as a tuple:

$$\tau_i = \langle c_i, m_i, s_i, D_i, \text{WP}_i \rangle \quad (12)$$

where c_i , m_i , and s_i denote the CPU, memory, and storage demands of task i ; D_i is the data size; and WP_i is the wireless access point associated with the device that generate the task.

On arrival, the scheduler performs a fast-path feasibility check at the Edge tier. Specifically, if the WAP Edge server s_E has sufficient resources to host the task, it is selected without utilizing PPo. The decision rule is based on projected utilization:

$$u_{\text{after}}^{\text{cpu}} = \frac{C^{\text{used}} + c_i}{C^{\text{total}}} \times 100\% \quad (13)$$

$$u_{\text{after}}^{\text{mem}} = \frac{M^{\text{used}} + m_i}{M^{\text{total}}} \times 100\% \quad (14)$$

If both projected utilizations remain at or below 80%, the task is immediately placed on s_E . This threshold is chosen because once a server exceeds roughly 80% utilization, its energy efficiency declines and its response performance begins to degrade, making further allocation undesirable.

If the WAP Edge cannot accommodate the task, the decision is delegated to a PPo agent. The policy selects one of two tiers:

$$a_i = \pi(s_i) \in \{ \text{PEaaS}, \text{Cloud} \} \quad (15)$$

Where s_i is the current system state encoding for task τ_i .

Once a tier is selected, the task is assigned to a specific server using deterministic logic. If PEaaS is chosen, then for the internal decision, the nearest feasible server (in terms of Euclidean distance to the WAP) is selected:

$$s^* = \arg \min_{s \in \mathcal{P}_{\text{feas}}} \|p_s - p_{\text{WP}}\| \quad (16)$$

If Cloud is selected, the server with the lowest propagation delay is chosen:

$$s^* = \arg \min_{s \in \mathcal{C}_{\text{feas}}} T_s^{\text{prop}} \quad (17)$$

This structured three-phase scheduling process (Edge short-circuit, PPo tier selection, and deterministic server selection) ensures that tasks are handled efficiently with minimal overhead while aligning with multi-tier constraints.

Reinforcement based scheduling

The core of the proposed scheduling logic relies on the RL agent that observes per-server metrics and makes tier-level offloading decisions. This section outlines the state representation, reward formulation, and PPo learning structure used during training and inference.

For each incoming task τ_i , the scheduler constructs a full observation of the system state by aggregating per-server features in the tier order. The feature vector for each server s with respect to task τ_i is:

$$\mathbf{x}_s(i) = [u_s^{\text{cpu}}, u_s^{\text{mem}}, g_s, \eta_s D_i + \kappa_s c_i, T_s^{\text{net}}(i), E_s(i), d_s^{\text{norm}}, \mathbb{I}_{\text{nearest}}(s)] \quad (18)$$

Here, u_s^{cpu} and u_s^{mem} denote the CPU and memory utilization of server s , respectively; g_s represents its congestion level; η_s is the transmission cost per KB, and κ_s is the processing cost per MIPS. The term D_i is the data size and c_i the CPU demand of task i . The cost component $\eta_s D_i + \kappa_s c_i$ represents the combined transmission and processing cost. $T_s^{\text{net}}(i)$ is the total network delay as defined in Section B, and $E_s(i)$ is the estimated energy required to offload task i to server s . The term d_s^{norm} denotes the normalized distance between server s and the originating wireless point, computed using the city grid diagonal. Finally, $\mathbb{I}_{\text{nearest}}(s)$ is an indicator function that equals 1 if server s is the nearest feasible PEaaS node.

The full state vector s_i is a concatenation of all server features:

$$\mathbf{s}_i = \bigoplus_{s \in \mathbb{T}} \mathbf{x}_s(i) \quad (19)$$

Here, s_i denotes the global state vector constructed for task i , while $\mathbf{x}_s(i)$ represents the feature vector of the server s as defined in Equation (15). The operator $\bigoplus$ denotes vector concatenation across all servers s in the set $\mathbb{T} = \mathbb{E} \cup \mathbb{P} \cup \mathbb{C}$, which includes all available Edge, PEaaS, and Cloud servers. This structured representation ensures stable server ordering and consistent state dimensionality, enabling effective learning by the PPo scheduler.

The scheduler receives feedback after each task decision through a composite reward function, covered in detailed in the Appendix A.

$$R_i = R_i^{\text{delay}} + R_i^{\text{cost}} + R_i^{\text{cong}} + R_i^{\text{energy}} + R_i^{\text{util}} + R_i^{\text{hier}} + R_i^{\text{over}} \quad (20)$$

The reward function is composed of several shaped components. The delay reward is defined as $R_i^{\text{delay}} = -\alpha_T T_{s^*}^{\text{net}}(i)$, where $T_{s^*}^{\text{net}}(i)$ is the total communication delay. The cost reward is given by $R_i^{\text{cost}} = -\alpha_C (\eta_{s^*} D_i + \kappa_{s^*} c_i)$, which penalizes higher transmission and processing costs. The congestion reward is $R_i^{\text{cong}} = -\alpha_g g_{s^*}$, discouraging allocation to heavily congested servers. The energy reward is defined as $R_i^{\text{energy}} = -\alpha_E E_{s^*}(i)$, penalizing higher energy consumption. Finally, the utilization shaping reward is given by $R_i^{\text{util}} = -\alpha_U (|u_{s^*}^{\text{cpu}} - u^*| + |u_{s^*}^{\text{mem}} - u^*|)$, which encourages server utilization to stay close to the optimal target u^* .

A hierarchy-aware bonus or penalty is applied to promote lower-tier placement:

$$R_i^{\text{hier}} = \begin{cases} -\beta_{\text{cloud}} & \text{if Cloud chosen and feasible PEaaS exists} \\ +\beta_{\text{PEaaS}} & \text{if PEaaS chosen and Edge over 68\%} \\ 0 & \text{otherwise} \end{cases} \quad (21)$$

Additionally, soft penalties are applied for over-utilization:

$$R_i^{\text{over}} = -\gamma_{\text{cpu}} \max(0, u_{s^*}^{\text{cpu}} - 80\%) - \gamma_{\text{mem}} \max(0, u_{s^*}^{\text{mem}} - 80\%) \quad (22)$$

This reward structure balances performance, cost, efficiency, and resource health.

The PPo agent adopts an actor-critic architecture composed of a shared multilayer encoder, followed by two task-specific heads for policy and value prediction. For each task i , the global state vector $s_i \in \mathbb{R}^d$ —formed by concatenating feature vectors across all candidate servers—is passed through a feed-forward neural network. The encoder produces a hidden representation h_i via the following transformation:

$$h_i = \text{ReLU}(W_2 \cdot \text{ReLU}(W_1 s_i + b_1) + b_2) \quad (23)$$

Here, $W_1 \in \mathbb{R}^{d_1 \times d}$ and $W_2 \in \mathbb{R}^{d_2 \times d_1}$ are trainable weight matrices of the first and second layers, respectively, while b_1 and b_2 are their corresponding bias vectors. The ReLU activation introduces non-linearity and ensures sparsity in the encoded features. The output vector $h_i \in \mathbb{R}^{d_2}$ serves as a shared latent representation for both policy and value estimation modules.

Following the shared encoder, the PPo agent branches into two separate output heads: a policy head (actor) and a value head (critic). The policy head produces action logits ℓ_i for task i over the available action space, which includes only the valid offloading tiers (PEaaS and Cloud). Simultaneously, the value head predicts the expected return V_i associated with the current state:

$$\ell_i = W_\pi h_i + b_\pi, \quad V_i = W_v h_i + b_v \quad (24)$$

Here, W_π and W_v are the respective weight matrices for the actor and critic layers, and b_π , b_v are their corresponding biases. The logits ℓ_i are passed through a softmax function to generate action probabilities for sampling or selection.

To update the policy, we adopt the clipped surrogate objective introduced by PPo. The overall loss function is defined as:

$$\mathcal{L}(\theta) = \mathbb{E}_t \left[-\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) + \lambda_V (V_t - R_t)^2 - \lambda_H \mathcal{H}(\pi_\theta) \right] \quad (25)$$

In this formulation, $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio between the new and old policy, and $\hat{A}_t = R_t - V_t$ is the advantage estimate. The first term inside the expectation maximizes the clipped surrogate objective, ensuring stable policy updates. The second term penalizes value prediction error, where λ_V is a scaling factor. The final term, weighted by λ_H , encourages exploration via the entropy $\mathcal{H}(\pi_\theta)$ of the policy distribution. During training, masking techniques are used to exclude invalid actions (e.g., selecting Cloud when PEaaS is feasible), ensuring compliance with hierarchical constraints.

Input: Task $\tau_i = \langle D_i, c_i, m_i, st_i \rangle$, assigned WAP edge server s_E , all servers $\mathbb{T} = \mathbb{E} \cup \mathbb{P} \cup \mathbb{C}$

Output: Selected server s^* or task failure

```

/* Using equations (Eq. (13), (14)) */
1 if  $s_E$  satisfies capacity constraints then
2   | Assign  $\tau_i$  to  $s_E$  and return
3 end
/* Determine feasible PEaaS and Cloud sets (Eq. (16), (17)) */
4 Prune  $\mathbb{P}$  and  $\mathbb{C}$  to  $\mathcal{P}_{\text{feas}}$ ,  $\mathcal{C}_{\text{feas}}$  based on capacity availability
5 if  $\mathcal{P}_{\text{feas}} = \emptyset$  and  $\mathcal{C}_{\text{feas}} = \emptyset$  then
6   | Mark task as failed and return
7 end
/* Construct global state vector  $\mathbf{s}_i$  (Eq. (19)) */
8  $\mathbf{s}_i \leftarrow \bigoplus_{s \in \mathbb{T}} \mathbf{x}_s(i)$  using Eq. (18)
/* Compute PPO logits and mask (Eq. (24)) */
9 Obtain logits  $\ell_i$  and value  $V_i$  from policy network
10 Apply tier availability masking to logits
11 Sample action  $a_i \in \{\text{PEaaS}, \text{Cloud}\}$ 
/* Server selection logic (Eq. (16), (17)) */
12 if  $a_i = \text{PEaaS}$  and  $\mathcal{P}_{\text{feas}} \neq \emptyset$  then
13   |  $s^* \leftarrow \arg \min_{s \in \mathcal{P}_{\text{feas}}} \|\mathbf{p}_s - \mathbf{p}_{\text{wp}}\|$ 
14 end
15 else if  $a_i = \text{Cloud}$  and  $\mathcal{C}_{\text{feas}} \neq \emptyset$  then
16   |  $s^* \leftarrow \arg \min_{s \in \mathcal{C}_{\text{feas}}} T_s^{\text{prop}}$ 
17 end
18 else
19   |  $s^* \leftarrow$  first available server from  $\mathcal{P}_{\text{feas}} \cup \mathcal{C}_{\text{feas}}$ 
20 end
/* Final feasibility and reward computation (Eq. (20)) */
21 if  $s^*$  can allocate  $\{c_i, m_i, st_i\}$  then
22   | Assign  $\tau_i$  to  $s^*$ 
23   | Compute  $R_i$  using reward function (Eq. (20)) and store experience
24 end
25 else
26   | Mark task as failed
27 end

```

Algorithm 1. PPO Based Tiered Offloading Scheduler

Algorithm

The PPO offloading algorithm, as shown in algorithm 1, decides on the most optimal tier to run the task τ_i . It receives the input task tuple $\tau_i = \langle D_i, c_i, m_i, st_i \rangle$, representing the data size, CPU, memory, and storage demands, respectively. The decision is made using the current system state $\mathbf{s}_i$, as defined in Equation (19), which encodes server metrics, task requirements, and network conditions.

When the task arrives, the algorithm first checks its feasibility at associated WAP s_E . If the CPU and memory utilization, calculated using Equations (13) and (14), remain below the 80% threshold, the task is directly placed on s_E .

If the Edge server is unable to serve the task due to resource constraints, the scheduler filters out infeasible servers from the PEaaS and Cloud tiers. These filtered server sets ($\mathcal{P}_{\text{feas}}$ and $\mathcal{C}_{\text{feas}}$) are computed based on the conditions listed in Equations (16) and (17). If no feasible server is found, the task is marked as failed.

Next, the algorithm constructs the full observation vector $\mathbf{s}_i$ by concatenating per-server features $\mathbf{x}_s(i)$ from Equation (18). These features include utilization levels, congestion, cost metrics, network delay, energy estimate, proximity, and nearest-tier indicators. This structured state representation is input to the PPO agent's policy network as presented in Equation (23).

The policy network outputs action logits and a value estimate. Logits corresponding to infeasible tiers are masked before applying the softmax operation. The action $a_i \in \{\text{PEaaS}, \text{Cloud}\}$ is sampled from the resulting probability distribution using the formulation in Equation (24).

Based on the selected action, the task is mapped to a specific server s^* . If PEaaS is selected, the nearest feasible server is chosen using Equation (16); if Cloud is selected, the server with the minimum propagation delay is chosen using Equation (17). If the selected tier has no valid server due to temporary fluctuations, the algorithm falls back to any remaining feasible server, ensuring task continuity.

The reward R_i associated with the decision is computed using the composite reward model as formulated in Equation (20), integrating delay, cost, congestion, energy, utilization alignment, tier hierarchy preference, and over-utilization penalties. These components guide the learning for better performance and efficiency optimized offloading.

The PPO loss function, formulated in Equation (25), is optimized using clipped surrogate objective, value loss, and entropy bonus. Experience tuples (s_i, a_i, R_i) are collected into mini-batches for gradient updates. Masking ensures that only valid tier selections contribute to the training process, maintaining hierarchical scheduling discipline.

Evaluation

We evaluate the proposed PEaaS offloading framework based on the *RegionalEdgeSimPy* simulator. We focus on all the necessary parameters, including latency, cost, distribution function related to tier selection, and demand, in terms of Edge, PEaaS layers, and the Cloud. We consider the effects related to device mobility, path loss, and handoffs between access points during the evaluation performed by the simulator.

Experimental setup

The simulation implements a three-tier hierarchy. The experimentation detailed configuration is shown in Table 2. The detailed configuration is also shown in Appendix B. Edge servers are co-located with WAP for local execution; PEaaS servers aggregate overflow from nearby edges; a remote Cloud datacenter provides additional capacity at increased latency and cost. We emulate an urban scenario with time-varying device positions, signal attenuation, and handovers. Workload intensity is scaled by incrementally increasing the number of active devices while keeping the task profile and network parameters fixed. The scheduler uses a PPO trained policy with a local-first preference. Tasks are executed on Edge until it reaches the upper utilization limit, then redirected to the nearest feasible PEaaS instance, with Cloud used only when PEaaS is over-utilized.

Results

CPU utilization across tiers is reported in Fig. 2. Workload is offloaded to the Edge first. As Edge utilization reaches the upper limit, the scheduler defers further placements on Edge and offloads to the nearest feasible PEaaS instance. The workload is offloaded to the cloud only when PEaaS is overutilized. At the Edge, utilization increases from 3.57 percent at round 1 to 78.57 percent by round 22. The trajectory crosses 70 percent at round 20 and 75 percent at round 21. After that, the next coming workload is transferred to the PEaaS (as shown in Fig. 2a). The mean Edge utilization over 300 rounds is 75.82 percent. At round 23 the workload is placed on the

Parameter	Setting
Number of servers	$ \mathbb{E} = 5, \mathbb{P} = 2, \mathbb{C} = 1$
Per-server CPU capacity (MIPS)	$(C_E, C_P, C_C) = 280\,000 / 6\,400\,000 / 100\,800\,000$
Per-server memory (MB)	$(M_E, M_P, M_C) = 300\,000 / 10\,240\,000 / 80\,640\,000$
Per-server storage (MB)	$(St_E, St_P, St_C) = 800\,000 / 40\,000\,000 / 1\,512\,000\,000$
Per-server bandwidth (Mbps)	$(B_E, B_P, B_C) = 164\,000 / 800\,000 / 1\,050\,000$
Baseline one-way latency (ms)	$(L_E, L_P, L_C) = 5 / 50 / 300$
Processing cost (\$/MIP)	$(\kappa_E, \kappa_P, \kappa_C) = 0.00005 / 0.00010 / 0.00020$
Transmission cost (\$/KB)	$(\eta_E, \eta_P, \eta_C) = 0.00002 / 0.000005 / 0.000005$
Nominal device-server distance (m)	$(d_E^{\text{nom}}, d_P^{\text{nom}}, d_C^{\text{nom}}) = 2\,000 / 200\,000 / 2\,000\,000$
City area (m)	$[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}] = [0, 10\,000] \times [0, 10\,000]$
Device count sweep	$ \mathcal{D} : 10 / 3\,000 / 10$ (start/max/step)
Default task data (KB)	$D^{\text{base}} = 5\,000$
Mobility (enable, step, ho-latency)	on, $\Delta t_{\text{mob}} = 5\,000$ ms, $T^{\text{ho}} = 5$ ms
Mobility speed (m/s)	$v_i(t) \in [3.0, 15.0]$, default 10.0
Scheduler policy	PPO (local-first, nearest feasible PEaaS)
CPU utilization limits (%)	$u_{\min}^{\text{cpu}} = 70, u_{\max}^{\text{cpu}} = 80$
Training hyperparameters	$p_{\text{explore}} = 0.30, \gamma = 0.99, K_{\text{epochs}} = 3$

Table 2. Simulation parameters.

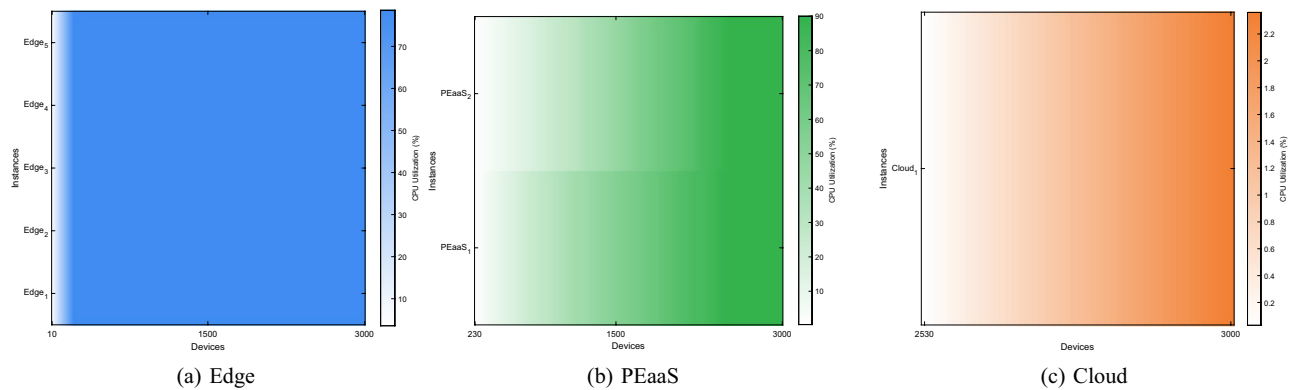


Fig. 2. CPU utilization by tier: (a) Edge, (b) PEaaS, (c) Cloud.

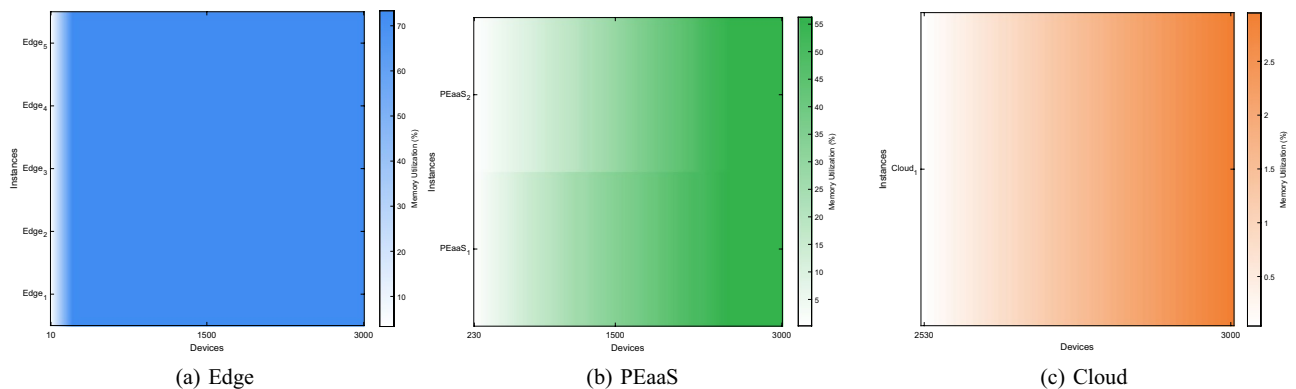


Fig. 3. Memory utilization by tier: (a) Edge, (b) PEaaS, (c) Cloud.

nearest feasible PEaaS instance. It increases from 0.39 percent at activation to 90 percent by round 253. Across rounds 23 to 300, the mean is 52.87 percent with a standard deviation of 29.11 percent, reflecting a ramp up phase followed by sustained high utilization that absorbs overflow from Edge (shown in Fig. 2b). For the Cloud tier, as we can see in Fig. 2c, utilization remains zero until PEaaS is near capacity. Cloud becomes active at round 253 and reaches a maximum of 2.36 percent by round 300, with a mean of 1.20 percent across the 48 active rounds. This confirms that cloud compute is reserved for overflow after PEaaS saturation.

Memory utilization across tiers is reported in Fig. 3. Workload is offloaded to the Edge first. As Edge memory reaches the upper limit, the scheduler defers further placements on Edge and offloads to the nearest feasible PEaaS. The workload is offloaded to the cloud only when PEaaS is overutilized. At the Edge, memory utilization rises with the initial devices and then stays around 74 to 75 percent for the remaining devices, which shows that new tasks are transferred away once the limit is reached (as shown in Fig. 3a). All Edge instances follow the same level across the range. At the PEaaS tier, memory utilization increases almost linearly as overflow is placed on the nearest feasible instance. The two PEaaS servers share the load; the small alternations between the bars reflect nearest selection. By the tail, both reach about 55 to 56 percent (as shown in Fig. 3b). For the Cloud tier, utilization remains near zero for most of the run and grows slowly when PEaaS is near capacity. It reaches about 3 percent at 3000 devices (as shown in Fig. 3c), which confirms that cloud memory is used only for overflow.

Storage utilization across tiers is reported in Fig. 4. Storage is not used as an input feature to the PPO scheduler; it varies with task placement decisions that are driven by CPU and memory utilization, congestion, delay, cost, and energy. Workload is offloaded to the Edge first. As Edge utilization reaches the upper limit, the scheduler defers further placements on Edge and offloads to the nearest feasible PEaaS instance. The workload is offloaded to the cloud only when PEaaS is overutilized. At the Edge, storage utilization rises quickly with the initial devices and then stays around 27 to 28 percent for the remaining devices, which shows that new tasks are transferred away once the limit is reached (as shown in Fig. 4a). All Edge instances follow the same level from 10 to 3000 devices. At the PEaaS tier, storage utilization increases almost linearly as overflow is placed on the nearest feasible instance. The two PEaaS servers share the load; the small alternations reflect nearest selection. By the tail, both reach about 14 to 15 percent (as shown in Fig. 4b). For the Cloud tier, utilization remains near zero for most of the run and grows slowly when PEaaS is near capacity. It reaches about 0.16 percent at 3000 devices (as shown in Fig. 4c), which confirms that cloud storage is used only for overflow.

Propagation delay across tiers is reported in Fig. 5. Devices transmit over wireless links to their WAP, so the one-way propagation time fluctuates with the device to WAP distance. When tasks remain at the Edge, the path

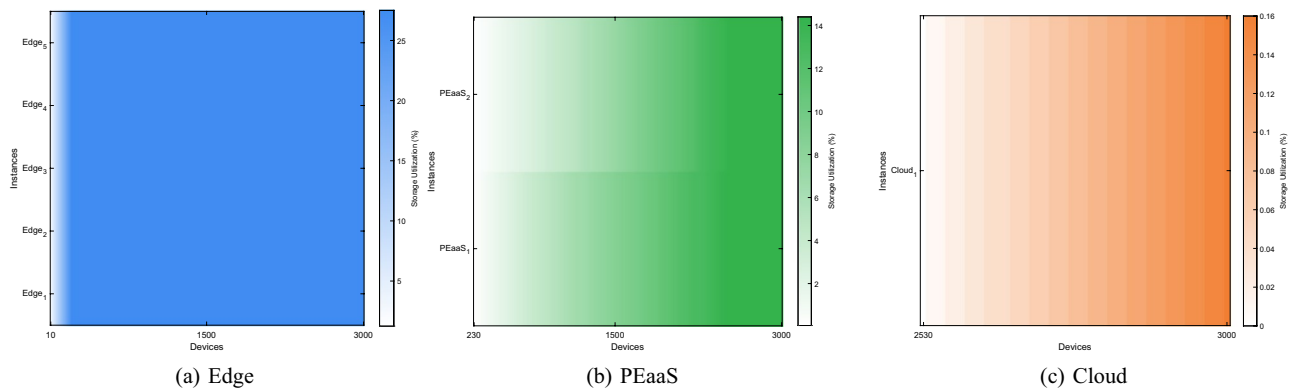


Fig. 4. Storage utilization by tier: (a) Edge, (b) PEaaS, (c) Cloud.

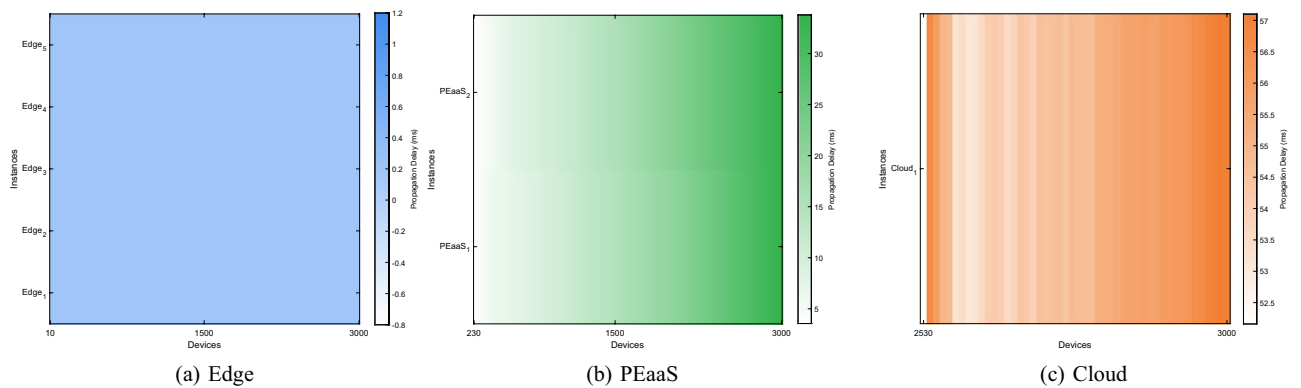


Fig. 5. Average propagation delay (ms) by tier: (a) Edge, (b) PEaaS, (c) Cloud.

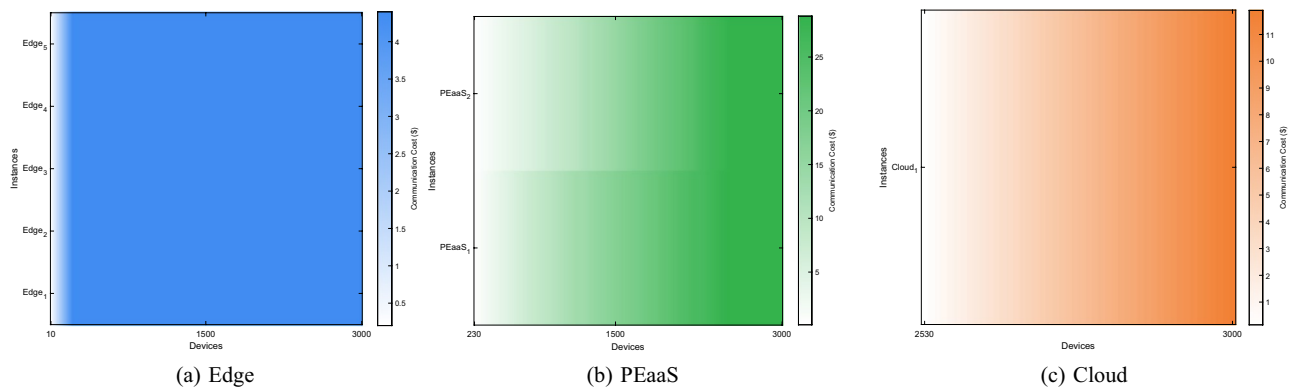


Fig. 6. Transmission cost by tier: (a) Edge, (b) PEaaS, (c) Cloud.

is shortest; offloading to PEaaS adds a regional hop; cloud execution introduces the longest path across the wide-area network. At the Edge, propagation delay remains near 0.2 ms across the full range, appearing essentially flat in Fig. 5a. Small ripples arise from changes in device and WAP separation under mobility, but computation co-located with the access point keeps the distance minimal. At the PEaaS tier, average propagation increases from about 3 to 4 ms at low device counts to roughly 34 to 35 ms near 3000 devices, with modest oscillations (Fig. 5b). This growth reflects traffic offloaded to the nearest feasible PEaaS instance and, over the range, the use of farther aggregators as edges serving different regions forward tasks; mobility-driven distance variation produces the observed fluctuations. For the Cloud tier, propagation is dominated by the datacenter distance and remains high and nearly flat, from about 46 to 60 ms with a slight upward drift (Fig. 5c). Only a small portion of tasks take this path late in the run; however, the long haul sets a higher propagation baseline than Edge or PEaaS.

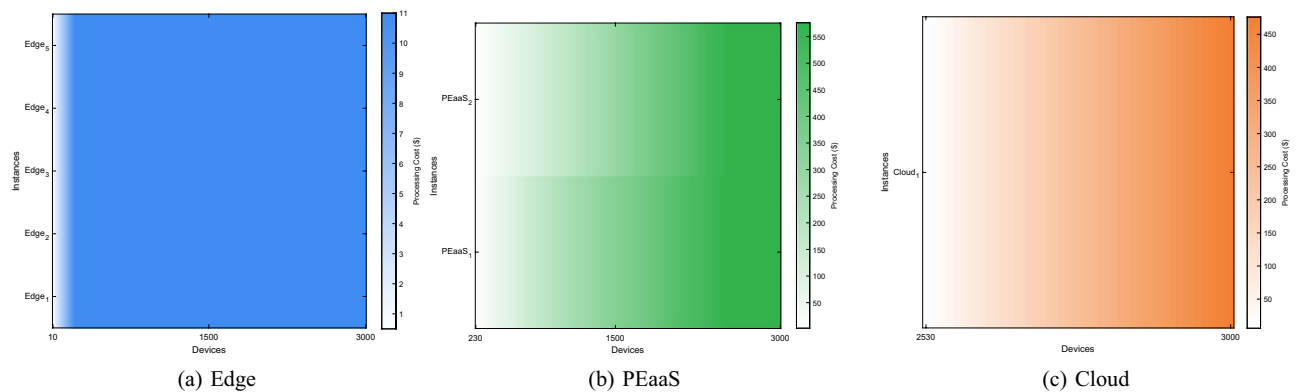


Fig. 7. Processing cost by tier: (a) Edge, (b) PEaaS, (c) Cloud.

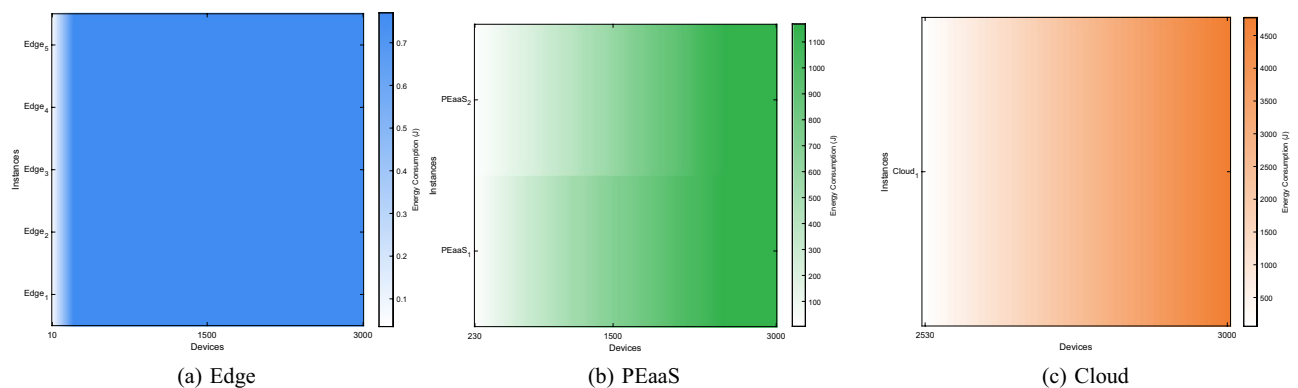


Fig. 8. Energy consumption by tier: (a) Edge, (b) PEaaS, (c) Cloud.

We report transmission cost by tier in Fig. 6. This metric reflects the cost of moving task data on the wireless interface to Edge, on the regional backhaul to the nearest feasible PEaaS instance, and on the wide area path to Cloud. The scheduler offloads to Edge first; when Edge reaches the upper limit, tasks are redirected to PEaaS; Cloud is used only when PEaaS is over-utilized. At the Edge, transmission cost rises with the initial devices and then stays around 4.1 to 4.3 for the remaining devices, which shows that new tasks are transferred away once the limit is reached (as shown in Fig. 6a). All Edge instances follow the same level from 10 to 3000 devices. At the PEaaS tier, transmission cost increases almost linearly as overflow is placed on the nearest feasible instance. The two PEaaS servers share the load; the small alternations reflect nearest selection. By the tail, both reach about 28 to 29 (as shown in Fig. 6b). For the Cloud tier, cost remains near zero for most of the run and grows slowly when PEaaS is near capacity. It reaches about 12 at 3000 devices (as shown in Fig. 6c), which confirms that cloud transmission is used only for overflow.

Processing cost across tiers is reported in Fig. 7. The scheduler places workload on the Edge first. When Edge utilization reaches the upper limit, further tasks are placed on the nearest feasible PEaaS instance. Cloud processing is used only when PEaaS is overutilized. At the Edge, processing cost rises quickly with the initial devices and then stays around 11 for the remaining devices, which shows that new tasks are transferred away once the limit is reached (as shown in Fig. 7a). All Edge instances remain at the same level from 10 to 3000 devices. At the PEaaS tier, processing cost increases almost linearly as overflow is placed on the nearest feasible instance. The two PEaaS servers share the load with small alterations that reflect the nearest selection. By the tail, both reach about 580 to 585 (as shown in Fig. 7b). For the Cloud tier, cost remains near zero for most of the run and grows as PEaaS approaches capacity. It reaches about 480 to 490 at 3000 devices (as shown in Fig. 7c), which confirms that cloud processing is reserved for overflow after PEaaS overutilization.

Energy consumption by tier is reported in Fig. 8. This metric captures compute energy drawn at each tier under the scheduler's placements. Workload is executed at the Edge first; when the Edge reaches the upper limit, tasks are placed on the nearest feasible PEaaS instance; Cloud energy appears only when PEaaS is overutilized. At the Edge, energy consumption rises quickly with the initial devices and then stays around 0.74 to 0.76 for the remaining devices, which shows that new tasks are transferred away once the limit is reached (as shown in Fig. 8a). All Edge instances remain at the same level from 10 to 3000 devices. At the PEaaS tier, energy consumption increases almost linearly as overflow is placed on the nearest feasible instance. The two PEaaS servers share the load; the small alternations reflect nearest selection. By the tail, both reach about 1160 to 1170 (as shown in Fig. 8b). For the Cloud tier, energy remains near zero for most of the run and grows as PEaaS

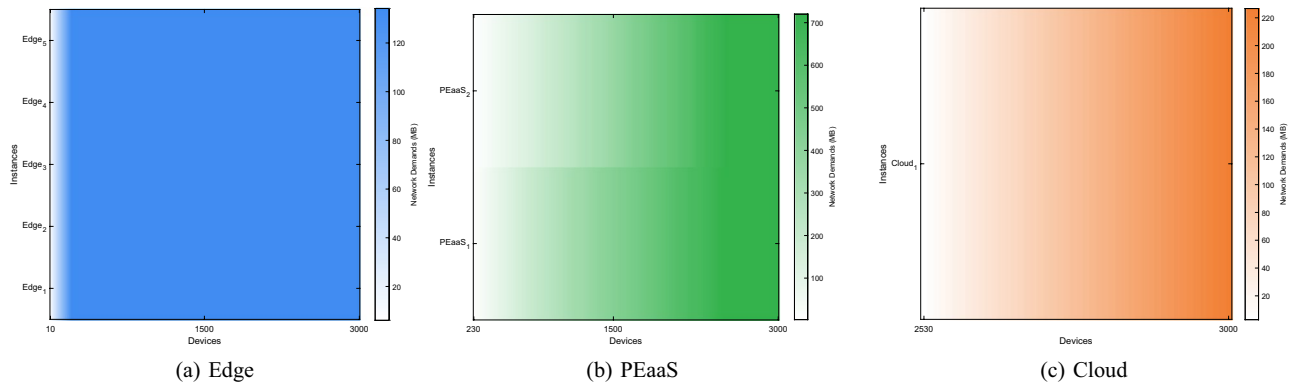


Fig. 9. Network demand by tier: (a) Edge, (b) PEaaS, (c) Cloud.

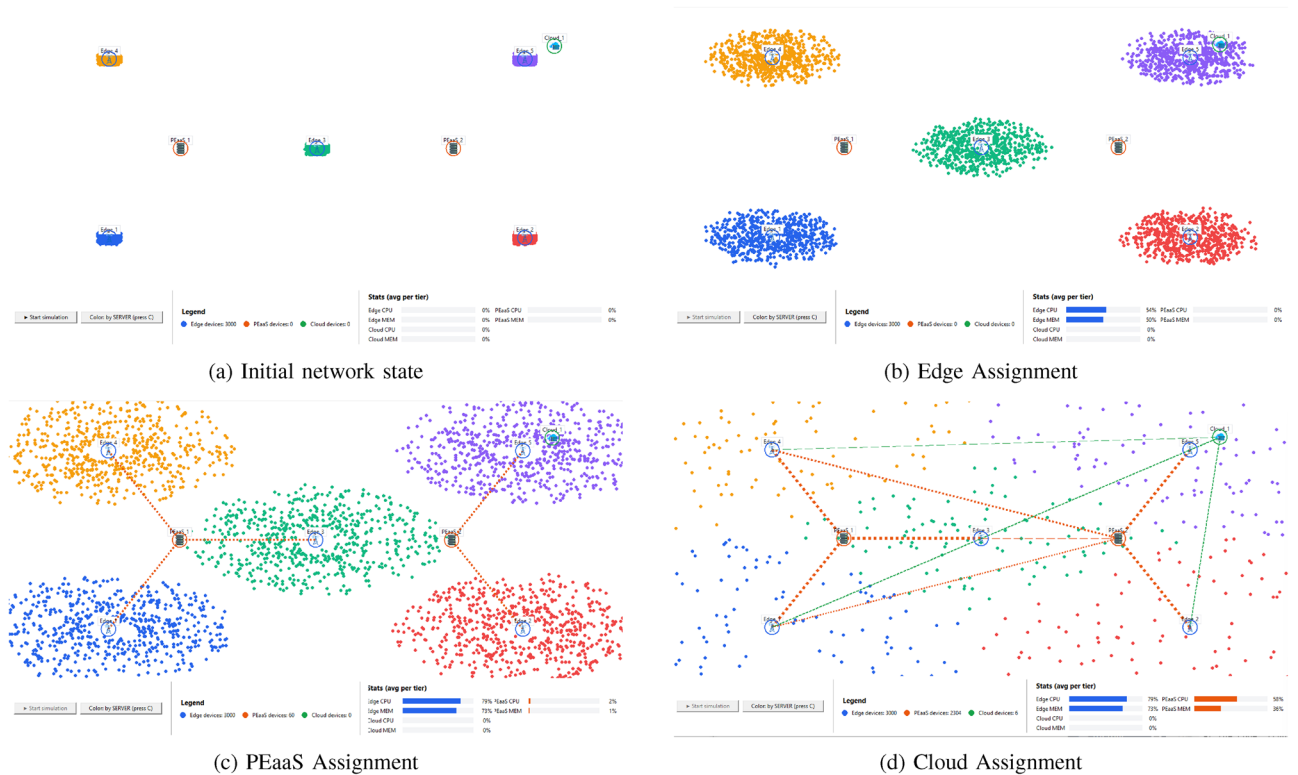


Fig. 10. Real-time RegionalEdgeSimPy snapshots across tiers: initial layout, edge placement, PEaaS aggregation, and cloud offloading.

approaches capacity. It reaches about 4800 to 4900 at 3000 devices (as shown in Fig. 8c), which confirms that cloud energy is used only for overflow.

We report network demand by tier in Fig. 9. This metric reflects the offered traffic load measured at each tier: the device to WAP, the Edge to PEaaS regional backhaul, and the Edge to Cloud wide area path. Traffic is served at the Edge first; when the Edge reaches the upper limit, overflow is routed to the nearest feasible PEaaS instance; Cloud is used only when PEaaS is overutilized. At the Edge, demand rises with the initial devices and then remains around 130 to 135 units for the remaining devices, which shows that new traffic is redirected once the limit is reached (as shown in Fig. 9a). All Edge sites track the same level from 10 to 3000 devices. At the PEaaS tier, demand increases almost linearly as overflow is carried on the regional backhaul to the nearest feasible instance. The two PEaaS servers share the load; small alterations reflect the nearest selection. By the tail, both reach about 720 to 725 units (as shown in Fig. 9b). For the Cloud tier, demand remains near zero for most of the run and grows as PEaaS approaches capacity. It reaches about 230 to 240 Mbps at 3000 devices (as shown in Fig. 9c), which confirms that the cloud path is used only for overflow.

Discussion

The stage-by-stage visualization in Fig. 10 and the aggregate metrics in Figs. 2,3,4,5,6,7,8,9 show that the proposed PEaaS policy preserves low latency while scaling to large device populations. It also extends capacity before any reliance on the cloud and localizes both cost and energy.

Latency is primarily proportional to the propagation distance. When tasks execute at the edge, propagation remains near 0.2 ms; when overflow is placed on the nearest feasible PEaaS instance, propagation increases to the regional range of about 3 to 35 ms; cloud execution remains much higher at about 46 to 60 ms (Fig. 5). Because placement is local first and then redirected to the nearest PEaaS, most tasks avoid the wide area path even at high load, so the delay distribution stays concentrated in the edge and regional ranges rather than the wide area range. This behavior matches the transition visible in Figure 10b–10c and the late appearance of cloud traffic in 10d.

Capacity is increased by holding the edge near its upper limit and absorbing additional demand at PEaaS. Edge CPU rises from 3.57 percent at the start to about 78.57 percent by round 22 and then remains near that level as new tasks are redirected. PEaaS CPU becomes active at round 23 and climbs to about 90 percent by round 253, while cloud CPU remains near zero until that point and peaks only near 2.36 percent (Fig. 2). Memory follows the same trend: edge memory settles around 74 to 75 percent, PEaaS memory grows with offered load and converges near 55 to 56 percent by the tail, and cloud memory stays negligible (Fig. 3). Storage shows a similar pattern with edge around 27 to 28 percent, PEaaS rising to about 14 to 15 percent, and cloud near zero (Fig. 4). These results indicate that the policy maintains useful headroom at the edge and defers cloud placements until both PEaaS instances are overutilized.

Cost results show that expenditure is localized where it delivers the largest delay reduction. Transmission cost at the edge is modest, about 4.1 to 4.3, and remains flat once redirection begins. PEaaS transmission cost increases with demand and reaches about 28 to 29 at the tail, while the cloud path used only for residual overflow reaches about 12 (Fig. 6). Processing cost exhibits the same hierarchy: edge cost remains low near 11, PEaaS rises with overflow to about 580 to 585, and cloud reaches about 480 to 490 at the maximum device count (Fig. 7). In practice, the scheduler spends on regional backhaul and aggregation where it buys latency, while keeping wide area transfers and remote compute limited to late spillover.

Energy measurements reinforce the same conclusion. PEaaS energy grows with redirected work and reaches about 1160 to 1170 by the tail, whereas cloud energy would be about 4800 to 4900 for the same device counts. Edge energy remains almost constant around 0.74 to 0.76 once redirection begins (Fig. 8). Concentrating heavy computation at nearby aggregation servers therefore yields a markedly lower energy footprint than sending the same overflow to a remote data center.

Network demand is steered away from the wide area backbone. Demand on the edge wireless path grows with the first devices and then stays around 130 to 135 Mbps once redirection starts. The regional backhaul to PEaaS carries the majority of incremental traffic and reaches about 720 to 725 Mbps at the tail, while the cloud path remains much lower, about 230 to 240 Mbps, because only residual workload is sent off site (Fig. 9). This redistribution of traffic explains why latency remains low for most tasks and why cost and energy remain localized even as the device population approaches the maximum.

These observations show that the PEaaS policy achieves low latency, higher effective capacity, localized cost and energy, and minimal dependence on the wide area network by enforcing local execution first, nearest PEaaS spillover under edge overutilization, and only late and limited use of the cloud.

Conclusion

This paper introduced (PEaaS) as a scalable framework that bridges the gap between edge and cloud computing. The work utilized a multi-tier architecture (Edge, PEaaS, and Cloud) and implemented it in the RegionalEdgeSimPy simulator. Through this setup, we analyzed the performance of big data communication and processing under realistic workload conditions. A RL scheduler based on PPo was developed to make offloading decisions using delay, cost, energy, utilization, and congestion as key parameters. The experimental results demonstrated that the proposed approach significantly improves performance compared to heuristic baselines. The scheduler adapts to changing conditions: it prioritizes edge resources when possible, shifts overflow to PEaaS, and uses the cloud as a last option. This leads to reduced latency, optimized cost, balanced utilization, and higher task success rates, confirming the effectiveness of PEaaS in maintaining low-latency and cost-efficient computation. Future work will focus on integrating task priority handling with mobility-aware scenarios. RegionalEdgeSimPy will also be enhanced for trace-driven validation to strengthen its applicability in real-world smart cities environments.

Data availability

The datasets and simulation code supporting the findings of this study will be made publicly available in a suitable online repository after the publication of this article.

Received: 6 October 2025; Accepted: 26 December 2025

Published online: 01 February 2026

References

1. Raza, S., et al. "An efficient task offloading scheme in vehicular edge computing," *Journal of Cloud Computing*, **9**, (2020).
2. Ullah, S. A., et al. "From nodes to roads: Surveying drl applications in mec-enhanced terrestrial wireless networks," *IEEE Communications Surveys & Tutorials*, (2025).
3. Sun, G. et al. Cost-efficient service function chain orchestration for low-latency applications in nfv networks. *IEEE Systems Journal* **13**(4), 3877–3888 (2018).
4. Huang, X., Xu, K., Lai, C., Chen, Q., & Zhang, J., "Energy-efficient offloading decision-making for mobile edge computing in vehicular networks," *Eurasip Journal on Wireless Communications and Networking*, **2020**, (2020).

5. Li, Y., et al., "Joint communication and offloading strategy of comp uav-assisted mec networks," *IEEE Internet of Things Journal*, (2025).
6. Ateeqa, J., et al., "Empowering edge computing: Public edge as a service for performance and cost optimization," *CMC-Computers, Materials and Continua*, (2025).
7. Song, L., Sun, G., Yu, H., & Niyato, D., "Espd-lp: Edge service pre-deployment based on location prediction in mec," *IEEE Transactions on Mobile Computing*, (2025).
8. Yang, C., Liu, Y., Chen, X., Zhong, W., & Xie, S., "Efficient mobility-aware task offloading for vehicular edge computing networks," *IEEE Access*, **7**, (2019).
9. Badshah, A., et al., "Smart cities' big data: performance and cost optimization with regional computing," *IEEE Access*, (2024).
10. Chen, Z., Hu, J., Min, G., Luo, C. & El-Ghazawi, T. Adaptive and efficient resource allocation in cloud datacenters using actor-critic deep reinforcement learning. *IEEE Transactions on Parallel and Distributed Systems* **33**(8), 1911–1923 (2021).
11. Badshah, A. et al. Network optimization by regional computing for uavs' big data. *Frontiers in Computer Science* **7**, 1565716 (2025).
12. Wu, T., et al., "Joint uav deployment and edge association for energy-efficient federated learning," *IEEE Transactions on Cognitive Communications and Networking*, (2025).
13. Yuan, W. et al. Transformer in reinforcement learning for decision-making: a survey. *Frontiers of Information Technology & Electronic Engineering* **25**(6), 763–790 (2024).
14. Li, Y.-F., Zhao, W., Zhang, C., Ye, J. & He, H. A study on the prediction of service reliability of wireless telecommunication system via distribution regression. *Reliability Engineering & System Safety* **250**, 110291 (2024).
15. Yao, L., Xu, X., Bilal, M., & Wang, H., "Dynamic edge computation offloading for internet of vehicles with deep reinforcement learning," *IEEE Transactions on Intelligent Transportation Systems*, **24**, (2023).
16. Shinde, S. S., Bozorgchenani, A., Tarchi, D., & Ni, Q., "On the design of federated learning in latency and energy constrained computation offloading operations in vehicular edge computing systems," *IEEE Transactions on Vehicular Technology*, **71**, (2022).
17. Hairul Nizam, M. H. Z., Ahmad Nizam, M. A., Jummadi, M. H. H. & N. N. M. S. Nik Mohd Kamal, and A. A. Zainuddin., Hyperledger fabric blockchain for securing the edge internet of things: A review, *Journal of Informatics and Web. Engineering* **4**(1), 81–98 (2025).
18. Jiang, H., et al., "Vehicle-assisted service caching for task offloading in vehicular edge computing," *IEEE Transactions on Mobile Computing*, (2025).
19. Xu, Y., et al., "Blockchain-based ar offloading in uav-enabled mec networks: A trade-off between energy consumption and rendering latency," *IEEE Transactions on Vehicular Technology*, (2025).
20. Shinde, S. S. & Tarchi, D. "A markov decision process solution for energy-saving network selection and computation offloading in vehicular networks," *IEEE Transactions on Vehicular Technology*, **72**, (2023).
21. Liu, J., et al., "A proactive stable scheme for vehicular collaborative edge computing," *IEEE Transactions on Vehicular Technology*, **72**, (2023).
22. Zhang, J., Guo, H., & Liu, J., "Adaptive task offloading in vehicular edge computing networks: a reinforcement learning based scheme," *Mobile Networks and Applications*, **25**, (2020).
23. Wang, C., Guan, Y., Peng, S., Chen, H., & Li, G., "Data prioritization aware resource allocation in internet of vehicles using multi-agent deep reinforcement learning," *Neural Networks*, 107671 (2025).
24. Meng, Q., Hussain, S., Luo, F., Wang, Z., & Jin, X., "An online reinforcement learning-based energy management strategy for microgrids with centralized control," *IEEE Transactions on Industry Applications*, (2024).
25. Karimi, E., Chen, Y., & Akbari, B., "Task offloading in vehicular edge computing networks via deep reinforcement learning," *Computer Communications*, **189**, (2022).
26. Liu, L., et al., "Mobility-aware multi-hop task offloading for autonomous driving in vehicular edge computing and networks," *IEEE Transactions on Intelligent Transportation Systems*, **24**, (2023).
27. Dai, Y. & Zhang, Y., "Adaptive digital twin for vehicular edge computing and networks," *Journal of Communications and Information Networks*, **7**, (2022).
28. Khawam, K., Khabbaz, M., & Saad, J., "A game theoretical priority-aware r2v task offloading framework for vehicular fog networks," *IEEE Open Journal of the Communications Society*, (2025).
29. Zhu, C., "Folo: Latency and quality optimized task allocation in vehicular fog computing," *IEEE Internet of Things Journal*, **6**, (2019).
30. Lin, B., et al., "Computation offloading strategy based on deep reinforcement learning for connected and autonomous vehicle in vehicular edge computing," *Journal of Cloud Computing*, **10**, (2021).
31. Moon, S., & Lim, Y., "Federated deep reinforcement learning based task offloading with power control in vehicular edge computing," *Sensors*, **22**, (2022).
32. Shi, W., Chen, L., & Zhu, X., "Task offloading decision-making algorithm for vehicular edge computing: A deep-reinforcement-learning-based approach," *Sensors*, **23**, (2023).
33. Tang, H., Wu, H., Qu, G., & Li, R., "Double deep q-network based dynamic framing offloading in vehicular edge computing," *IEEE Transactions on Network Science and Engineering*, **10**, (2023).
34. Chen, Z. et al., "Resilient collaborative caching for multi-edge systems with robust federated deep learning," *IEEE/ACM Transactions on Networking*, (2024).
35. Zhao, L., et al., "Meson: A mobility-aware dependent task offloading scheme for urban vehicular edge computing," *IEEE Transactions on Mobile Computing*, **23**, (2024).
36. Chen, Z. & Yu, Z. Intelligent offloading in blockchain-based mobile crowdsensing using deep reinforcement learning. *IEEE Communications Magazine* **61**(6), 118–123 (2023).
37. Wang, J., Lv, T., Huang, P., & Mathiopoulos, P. T., "Mobility-aware partial computation offloading in vehicular networks: A deep reinforcement learning based scheme," *China Communications*, **17**, (2020).
38. Ling, C., et al., "Qos and fairness oriented dynamic computation offloading in the internet of vehicles based on estimate time of arrival," *IEEE Transactions on Vehicular Technology*, **73**, (2024).
39. Yan, J., Zhao, X., & Li, Z., "Deep-reinforcement-learning-based computation offloading in uav-assisted vehicular edge computing networks," *IEEE Internet of Things Journal*, **11**, (2024).
40. Min, H., Rahmani, A. M., Ghaderkourehpaz, P., Moghaddasi, K. & Hosseinzadeh, M. A joint optimization of resource allocation management and multi-task offloading in high-mobility vehicular multi-access edge computing networks. *Ad Hoc Networks* **166**, 103656 (2025).
41. He, W., et al., "A deep reinforcement learning approach to time delay differential game deception resource deployment," *IEEE Transactions on Dependable and Secure Computing*, (2025).
42. Guan, P. et al. Context-aware container orchestration in serverless edge computing In 1041–1046 (2024).
43. Chen, C., et al., "Cost optimization for serverless edge computing with budget constraints using deep reinforcement learning," *arXiv preprint arXiv:2501.12783*, (2025).
44. Maleki, H., Başaran, M., & Durak-Ata, L., "Handover-enabled dynamic computation offloading for vehicular edge computing networks," *IEEE Transactions on Vehicular Technology*, **72**, (2023).
45. Ju, Y., et al., "Joint secure offloading and resource allocation for vehicular edge computing network: A multi-agent deep reinforcement learning approach," *IEEE Transactions on Intelligent Transportation Systems*, **24**, (2023).

46. Hao, H., Xu, C., Zhang, W., Yang, S., & Muntean, G. M., “Task-driven priority-aware computation offloading using deep reinforcement learning,” *IEEE Transactions on Wireless Communications*, (2025).

Author contributions

All authors contributed significantly to this work: Ateeqa Jalal and Afzal Badshah conceptualized and designed the study; Afzal Badshah, Ateeqa Jalal, and Ihsan Rabbi developed the simulation framework and reinforcement learning scheduler; Umar Farooq and Aurangzeb Khan performed experiments and data validation; Umar Farooq, Aurangzeb Khan, Muhammad Mansoor Alam, and Mazliham Mohd Su’ud worked on results and contributed to critical review and editing; and Mazliham Mohd Su’ud and Afzal Badshah supervised the overall research and project administration.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Supplementary Information The online version contains supplementary material available at <https://doi.org/10.1038/s41598-025-34308-5>.

Correspondence and requests for materials should be addressed to A.B. or M.M.S.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher’s note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article’s Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article’s Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025